

Sesiunea 1: Arhitectura web, structura DOM-ului și fundamente HTML

Capitolul 1: Arhitectura Web - Cum funcționează Internetul și Browserele	2
Obiective de învățare	2
1.1 De la QA Manual la Automation: Schimbarea de perspectivă	3
1.2 Arhitectura de bază: Client - Server	3
1.3 Ciclul de viață al unei cereri: HTTP Request & Response	3
1.4 De ce este acest lucru critic pentru un QA Engineer?.....	4
1.5 Știați că...?	4
1.6 Poveste Aplicată: „Restaurantul Digital”	5
1.7 Exerciții Practice	5
1.8 Răspunsuri la Întrebări & Soluții la Exerciții	5
Capitolul 2: Introducere în HTML și Tag-uri Semantice	6
Obiective de învățare	6
2.1 Ce este HTML-ul? (Nu este un limbaj de programare!)	6
2.2 Anatomia unui Element HTML: Tag-uri și Conținut.....	6
2.3 Structura de bază a unui document HTML (Boilerplate)	7
2.4 HTML Semantic: De ce contează pentru noi, ca tester?	8
2.5 Atributele HTML: Ancorele Automatizării	8
2.6 Știați că...?	10
2.7 Poveste Aplicată: „Fundatia și Cărmizile Casei”	10
2.8 Exerciții Practice	11
2.9 Răspunsuri la Întrebări & Soluții la Exerciții	11
Capitolul 3: Structura DOM-ului și Workshop Practic (Task Tracker)	12
Obiective de învățare	13
3.1 Ce este DOM-ul (Document Object Model)?	13
3.2 Arborele Genealogic: Părinți, Copii și Frați	13

3.3 De ce este DOM-ul „Terenul de joacă” al automatizării?	14
3.4 Cum „citește” Playwright DOM-ul? (O privire sub capotă).....	15
3.5 Workshop Practic: Crearea scheletului extins pentru aplicația „Task Tracker”15	
3.6 Știați că...?	18
3.7 Poveste Aplicată: „Arborele Genealogic al Familiei Web”	18
3.8 Exerciții Practice	18
3.9 Răspunsuri la Întrebări & Soluții la Exerciții	19
Capitolul 4: Memoria Browserului, Metode HTTP și Tema Sesiunii 1	20
Obiective de învățare	20
4.1 Metode HTTP: GET vs. POST (Ce se întâmplă cu datele din formular?).....	21
4.2 Memoria Browserului: Cookies, Local Storage și Session Storage	21
4.3 DevTools: Tab-ul Application (Arma secretă a QA-ului)	22
4.4 Știați că...?	23
4.5 Poveste Aplicată: „Cartea Poștală și Coletul Blindat”	23
4.6 Exerciții Practice	23
4.7 Răspunsuri la Întrebări & Soluții la Exerciții	24
4.8 Tema pentru Acasă (Proiect Sesiunea 1) & Platforma QualiAdept Cloud Evaluation.....	24
Introducere în Platforma Certify QualiAdept.....	24
Cum te Auto-Validezi (Predarea Temei)	25

Capitolul 1: Arhitectura Web - Cum funcționează Internetul și Browserele

Obiective de învățare



La finalul acestui capitol, vei fi capabil să:

- Explici diferența fundamentală dintre interfața utilizatorului (Frontend) și server (Backend).
- Înțelegi conceptele de Client, Server și Bază de Date.
- Urmărești traseul unei cereri HTTP de la apăsarea tastei Enter până la afișarea site-ului.
- Folosești tab-ul "Network" din DevTools pentru a intercepta primul tău Request.

1.1 De la QA Manual la Automation: Schimbarea de perspectivă

În testarea manuală, interacționezi cu aplicația exact ca un utilizator final: dai click pe butoane, completezi formulare și verifici dacă rezultatul vizual este cel corect (Black Box Testing).

Ca viitor **QA Automation Engineer**, trebuie să mergi un nivel mai profund. Când un test automatizat eșuează (ex: Playwright nu găsește un buton), trebuie să știi *de ce* a eșuat. Pentru asta, trebuie să înțelegi drumul pe care îl parcurge codul de pe server până pe ecranul tău. Nu mai testăm doar "ceea ce se vede", ci testăm infrastructura.

1.2 Arhitectura de bază: Client - Server

Aproape orice aplicație web modernă (inclusiv aplicația "Task Tracker" pe care o vom construi) funcționează pe baza arhitecturii **Client-Server**.

- **Clientul:** Este aplicația care cere informații. De cele mai multe ori, clientul este **Browser-ul web** (Chrome, Firefox, Safari) de pe laptopul sau telefonul tău. Când vom scrie scripturi de automatizare, programul nostru Playwright va acționa ca un "client robot".
- **Serverul:** Este un calculator puternic, conectat non-stop la internet, care "servește" date. Aici locuiește "creierul" aplicației (Backend-ul) și aici sunt stocate fișierele (HTML, CSS, imagini).
- **Baza de date (Database):** Este "dulapul cu dosare" al serverului. Aici sunt stocate informațiile pe termen lung (ex: conturile utilizatorilor, parolele criptate, task-urile salvate).

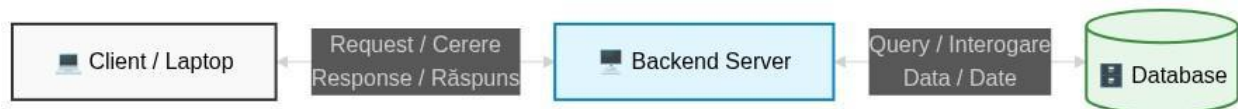


Fig.1 - Diagramă de arhitectură 3-Tier. Săgețile bidirecționale indică comunicarea sincronă între interfața utilizatorului (Frontend), logica aplicației (Backend) și nivelul de persistență (Database).

1.3 Ciclul de viață al unei cereri: HTTP Request & Response

Comunicarea dintre Client și Server se face printr-un set de reguli numit **Protocolul HTTP** (HyperText Transfer Protocol). Gândește-te la HTTP ca la limba comună pe care o vorbesc ambele calculatoare.

Iată ce se întâmplă, pas cu pas, când scrii `www.emag.ro` în browser și apeși Enter:

- **HTTP Request (Cererea):** Browserul tău (Clientul) trimite un mesaj către serverul eMAG. Mesajul spune: *"Salut, dă-mi te rog pagina principală"*. Acest mesaj se numește cerere de tip GET.
- **Procesarea pe Server:** Serverul primește mesajul. Se uită în Baza de Date pentru a lua cele mai noi produse, assemblează o pagină web și o pregătește de livrare.
- **HTTP Response (Răspunsul):** Serverul trimite înapoi un "pachet" către browser. Pachetul conține:
 - Un **Status Code** (ex: 200 OK - totul e perfect, sau 404 Not Found - pagina nu există).
 - **Codul HTML** (structura paginii).
 - **CSS și JavaScript** (designul și logica vizuală).
- **Randarea (Afișarea):** Browserul tău primește pachetul, citește codul de sus în jos și "desenează" butoanele și imaginile pe ecranul tău.

1.4 De ce este acest lucru critic pentru un QA Engineer?

Dacă un utilizator dă click pe "Login" și ecranul rămâne blocat, un QA Manual va scrie un bug: *"Butonul de login nu merge"*.

Un QA Automation care înțelege arhitectura va deschide **DevTools**, va vedea că Request-ul către server a plecat, dar Serverul a returnat 500 Internal Server Error. Bug-ul raportat va fi: *"Serverul returnează eroarea 500 la accesarea endpoint-ului de login"*. Primul raport este vag; al doilea ajută dezvoltatorul să repare problema în 5 minute.

1.5 Știați că...?

💡 Știați că...

- **Pachetele pierdute:** Datele trimise prin internet nu călătoresc ca o singură bucată mare. Ele sunt tăiate în mii de "pachete" mici, care pot lua rute diferite prin lume. Browserul tău le reassemblează la destinație.
- **Status Codes de bază:** Există o regulă universală pentru codurile de răspuns HTTP pe care orice QA trebuie să le știe:
 - 2xx - Succes (Totul a mers bine).
 - 3xx - Redirect (Pagina a fost mutată).
 - 4xx - Eroare la Client (Tu ai greșit, ex: ai căutat un URL care nu există - 404).

- 5xx - Eroare la Server (A crăpat serverul).

1.6 Poveste Aplicată: „Restaurantul Digital”

Cea mai bună analogie pentru arhitectura Client-Server este un restaurant.

- **Tu ești Clientul (Browserul):** Stai la masă și te uiți pe meniu (Interfața - UI).
- **Chelnerul este HTTP Request-ul:** Tu îi spui chelnerului: *"Vreau o pizza"*. El ia comanda ta și o duce la bucătărie.
- **Bucătăria este Serverul (Backend):** Bucătarul ia comanda, strânge ingredientele, prepară pizza și aplică regulile de business (ex: "Fără ceapă").
- **Cămara cu alimente este Baza de Date:** De acolo ia bucătarul ingredientele brute.
- **Chelnerul se întoarce (HTTP Response):** Îți aduce pizza la masă împreună cu un status ("Poftă bună!" = 200 OK, sau "Nu mai avem blat" = 404 Not Found).

Ca Automation Tester, treaba ta nu este doar să guști pizza (Testare UI), ci uneori să interceptezi chelnerul pe drum (API Mocking) pentru a vedea dacă a notat comanda corect.

1.7 Exerciții Practice

Exercițiul 1: Interceptarea propriului Request (Inspectorul Network)

Vom folosi arma principală a oricărui inginer QA: Browser DevTools.

- Deschide Google Chrome.
- Dă click-dreapta oriunde pe o pagină goală și alege **"Inspect"** (sau apasă F12 / Ctrl+Shift+I).
- În meniul de sus al panoului care s-a deschis, dă click pe tab-ul **Network** (Rețea).
- Scrie în bara de adrese a browserului www.wikipedia.org și apasă Enter.
- *Sarcina ta:* Privește cascada de fișiere care apar în tab-ul Network. Găsește primul fișier din listă (de obicei se numește wikipedia.org). Dă click pe el și identifică în secțiunea "Headers" care este **Status Code-ul** primit.

1.8 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1:

Dacă ai urmat pașii corect, primul element din tab-ul Network reprezintă Documentul HTML principal. Când dai click pe el, în panoul lateral din dreapta, sub secțiunea **General**, vei vedea

câmpul Status Code. Ar trebui să fie 200 OK (verde).

 *Felicitări! Tocmai ai interceptat și analizat cu succes prima ta tranzacție Client-Server. Acest obicei te va salva de nenumărate ore de frustrare când vom face debugging pe testele noastre automatizate.*

Capitolul 2: Introducere în HTML și Tag-uri Semantice

Obiective de învățare

-  La finalul acestui capitol, vei fi capabil să:
- Explici rolul limbajului HTML în construirea unei pagini web.
 - Scrii corect elemente HTML, respectând sintaxa de deschidere și închidere.
 - Înțelegi diferența și importanța atributelor critice pentru automatizare (id, class, data-testid).
 - Construiești o pagină web simplă, structurată semantic, gata să fie interceptată de un script Playwright.

2.1 Ce este HTML-ul? (Nu este un limbaj de programare!)

Este o confuzie frecventă la început de drum. **HTML (HyperText Markup Language)** NU este un limbaj de programare. Nu poți scrie cu el o logică de tipul „dacă $2+2=4$, atunci afișează o alertă”.

HTML este un **limbaj de marcare**. Rolul său unic și exclusiv este de a structura informația, spunându-i browserului ce reprezintă fiecare bucată de text: „Acesta este un titlu major”, „Acesta este un paragraf”, „Aici avem o listă” sau „Acesta este un buton pe care se poate da click”.

Ca QA Automation Engineer, HTML-ul este harta ta. Dacă nu știi să citești harta, robotul tău (Playwright) se va rătăci.

2.2 Anatomia unui Element HTML: Tag-uri și Conținut

Pentru a „marca” textul, HTML folosește **Tag-uri** (etichete), scrise mereu între paranteze

unghiulare < >.

Majoritatea elementelor HTML au o structură din 3 părți:

- **Tag-ul de deschidere:** Marchează începutul elementului. Ex: <button>
- **Conținutul:** Ceea ce va vedea efectiv utilizatorul pe ecran. Ex: Trimite Comanda
- **Tag-ul de închidere:** Marchează sfârșitul elementului. Se distinge prin adăugarea unui slash / (bară oblică). Ex: </button>

Element complet: <button>Trimite Comanda</button>



Fig. 2 - Anatomia unui element HTML. Descompunerea etichetei <p> în tag de start (verde), conținut (albastru) și tag de închidere (roșu), evidențiind bara oblică (/).

Atenție (Elemente cu auto-închidere):

Există anumite elemente care nu pot conține text și, prin urmare, nu au nevoie de un tag separat de închidere. Cele mai comune în QA sunt imaginile și câmpurile de completat (input-urile):

- ``
- `<input type="text" />`

2.3 Structura de bază a unui document HTML (Boilerplate)

Orice fișier HTML valid din lume respectă un schelet (boilerplate) standard. Iată cum arată și ce

reprezintă fiecare zonă:

```
<!DOCTYPE html> <!-- 1. Declarația tipului de document: Îi spune browserului că folosim HTML5 (cea mai nouă versiune). -->
<html lang="ro"> <!-- 2. Elementul Rădăcină (Root): Tot codul stă înăuntrul acestui tag. -->

    <head> <!-- 3. Partea de "Creier" (Metadata) -->
        <!-- Aici stau informații PENTRU browser, NU pentru utilizator. Nimic de aici nu e vizibil pe pagina albă (excepție: titlul din tab-ul de sus). -->
        <meta charset="UTF-8">
        <title>Aplicația Mea QA</title>
    </head>

    <body> <!-- 4. Partea "Vizibilă" -->
        <!-- Aici pui ABSOLUT TOT ce vrei să vadă utilizatorul pe ecran și tot ce vei testa automatizat: butoane, formulare, texte, imagini. -->
        <h1>Bine ai venit la Task Tracker!</h1>
        <p>Aici vom adăuga task-urile noastre.</p>
    </body>

</html>
```

2.4 HTML Semantic: De ce contează pentru noi, ca testerii?

În anii 2000, dezvoltatorii foloseau un tag generic numit <div> pentru a împărți pagina (ex: <div id="sus">, <div id="jos">). Era un haos greu de citit.

HTML5 a introdus **Tag-urile Semantice**. Acestea sunt etichete care își descriu clar rolul. Acestea ajută enorm motoarele de căutare (SEO), softurile pentru persoanele cu deficiențe de vedere, dar și pe **noi (QA Engineers)**, pentru că fac codul sursă infinit mai ușor de investigat:

- <header>: Antetul paginii sau al unei secțiuni (unde stă logo-ul).
- <nav>: O secțiune care conține link-uri de navigare (Meniul).
- <main>: Conținutul principal și unic al paginii.
- <section>: O secțiune tematică generică.
- <footer>: Subsolul paginii (unde stă textul de Copyright sau link-urile de contact).

2.5 Atributele HTML: Ancorele Automatizării

Dacă Tag-urile spun ce este un element, **Atributele** oferă *informații suplimentare* despre acel element. Atributele se scriu **întotdeauna în tag-ul de deschidere**.

Sintaxa este: `nume_atribut="valoare"`.

Acesta este cel mai important concept din acest capitol pentru viitorul tău rol de QA Automation. Aceste atribute vor fi "cârligele" de care te vei agăța în codul tău Playwright pentru a găsi butoanele pe pagină.

Atribut	Exemplu	Explicație & Relevanță pentru QA
id	<code><button id="login-btn"></code>	CRITIC! ID-ul trebuie să fie unic (nu se repetă niciodată pe aceeași pagină). Este cel mai rapid, stabil și preferat mod de a localiza un element (ex: <code>page.locator("#login-btn")</code>).
class	<code><p class="error-text"></code>	Definește o "clasă" de elemente, folosită de obicei pentru a le colora la fel prin CSS. Mai multe elemente pot avea aceeași clasă. Îl folosim pentru a extrage liste de elemente.
type	<code><input type="checkbox"></code>	Ne spune ce fel de input este (text, parolă, bifă, buton radio). Foarte util când avem zeci de input-uri.
name	<code><input name="email"></code>	Folosit frecvent la formulare pentru a trimite datele către

		server. Un localizator excelent în QA.
data-*	<code><button data-testid="submit-login"></code>	Sfântul Graal al Automatizării! (ex: data-testid, data-qa). Sunt attribute personalizate. Dezvoltatorii buni le pun special pentru noi, ca testele noastre să nu pice dacă cineva schimbă designul (clasele CSS).

2.6 Știați că...?

💡 Știați că...

- **HTML nu este "Case Sensitive":** Browserului nu îi pasă dacă scrii `<BUTTON>`, `<Button>` sau `<button>`. Totuși, standardul global, strict recomandat în industrie (și pe care îl vom folosi), este scrierea exclusivă cu litere mici (lowercase).
- **Id-urile duplicate strică testele automatizate:** Dacă un developer greșește și pune `id="submit"` pe două butoane diferite, browserul Chrome este iertător și va afișa pagina fără erori vizibile. Dar scriptul tău de Playwright se uită în DOM, vede primul element cu acel ID, dă click pe el și **îl ignoră complet pe al doilea**. O mulțime de "flaky tests" (teste instabile) provin din această greșeală a developerilor!

2.7 Poveste Aplicată: „Fundația și Cărmizile Casei”

Dacă te gândești la o pagină web ca la o casă:

- **Structura Boilerplate** (`<html>`, `<head>`, `<body>`) reprezintă fundația, acoperișul și pereții exteriori. Nu poți avea o casă fără ele.
- **Tag-urile HTML** (`<h1>`, `<p>`, `<button>`) sunt cărmizile, ferestrele și ușile. Ele spun ce e

acolo: "Aici punem o ușă".

- **Atributele** (id, class) sunt etichetele pe care le lipești pe aceste elemente pentru echipa de control. Dacă îi spui unui muncitor (scriptului de test) "Verifică dacă ușa e închisă", el va întreba "Care ușă? Sunt 10!". Dacă îi spui "Verifică elementul cu id='usa-intrare-principala'", se va duce direct la țintă, fără nicio confuzie.

În testarea automatizată, tu vei fi inspectorul care trimite roboții să verifice "etichetele" (atributele) pe șantier.

2.8 Exerciții Practice

Exercițiul 1: Vânătoarea de Attribute (DevTools)

Să folosim cunoștințele proaspete în lumea reală.

- Deschide Google Chrome și intră pe www.emag.ro (sau orice site mare).
- Dă click dreapta pe bara principală de căutare și apasă **"Inspect"** (sau Inspectează).
- În panoul Elements, codul se va lumina pe rândul corespunzător barei de căutare.
- *Misiunea ta:* Identifică și notează undeva ce **id** are acel `<input>` de căutare. Dar atributul **type**?

Exercițiul 2: Scrie propriul cod Semantic

Deschide un editor de text (ex: Notepad sau VS Code, fără a rula încă în browser) și scrie un bloc de cod HTML (doar ce ar veni în interiorul `<body>`), care să conțină:

- O zonă `<header>` în care să pui un titlu principal `<h1>` cu textul "Magazinul Meu".
- O zonă `<main>` în care să ai un formular cu:
 - Un `<input>` de tip text, cu `id="search-box"`.
 - Un `<button>` care să aibă atributul specific automatizării `data-testid="search-btn"` și textul "Caută Produs".

2.9 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1:

Deși site-urile se pot actualiza, de regulă la eMAG (sau site-uri similare), bara de căutare va fi un tag `<input>`. Vei observa că are un atribut de tipul `type="search"` sau `type="text"`. ID-ul este de obicei foarte descriptiv, de genul `id="searchboxTrigger"` sau simplu `id="search"`. Așa l-ai localiza într-un test automatizat!

Soluție Exercițiul 2:

Iată cum arată un cod scris "ca la carte", respectând standardele pe care ne vom baza în Playwright:

```
<header>
  <h1>Magazinul Meu</h1>
</header>
<main>
  <input type="text" id="search-box">
  <button data-testid="search-btn">Caută Produs</button>
</main>
```

Dacă ai reușit să scrii acest snippet corect, cu parantezele unghiulare și ghilimelele la locul lor, ești gata să intri adânc în Structura DOM-ului în capitolul următor!

Capitolul 3: Structura DOM-ului și Workshop Practic (Task Tracker)

Obiective de învățare

 La finalul acestui capitol, vei fi capabil să:

- Explici ce este Document Object Model (DOM) și cum diferă acesta de codul HTML brut.
- Identifici relațiile de rudenie (Părinte-Copil-Frațe) între elementele web dintr-o pagină.
- Înțelegi exact cum interacționează Playwright direct cu structura DOM-ului, ocolind interfața grafică clasică.
- Construiești o interfață HTML complexă (formulare, selecții, tabele) pentru aplicația web "Task Tracker", adăugând attribute de testabilitate.

3.1 Ce este DOM-ul (Document Object Model)?

Dacă HTML-ul este codul sursă (textul) trimis de server, **DOM-ul (Document Object Model)** este reprezentarea "vie" a acelui cod, construită de browser în memoria sa.

Când browserul (Chrome, Edge) primește fișierul .html, el nu afișează direct textul tău. El ia acel text, îl parsează (îl citește bucată cu bucată) și îl transformă într-o structură de date interactivă, asemănătoare unui arbore. Acel arbore este DOM-ul.

Odată creat DOM-ul, browserul îl folosește pentru a "desena" (randa) pagina pe ecran. Mai târziu, când vom folosi JavaScript, vom putea modifica DOM-ul în timp real (de exemplu, adăugând un element nou pe ecran fără să reîncărcăm pagina).

3.2 Arborele Genealogic: Părinți, Copii și Frați

Pentru a putea localiza elemente mai târziu cu Playwright (mai ales când nu avem ID-uri utile), trebuie să înțelegem cum sunt "înrudite" elementele în DOM. DOM-ul este o ierarhie strictă:

- **Root (Rădăcina):** Este punctul de plecare, mereu elementul `<html>`.
- **Parent (Părinte):** Un element care conține direct alt element. De exemplu, `<body>` este părintele tuturor elementelor vizibile de pe pagină.
- **Child (Copil):** Un element aflat direct în interiorul altui element. Un `<h1>` pus în interiorul unui `<header>` este copilul header-ului.
- **Siblings (Frați):** Elemente care împart exact același părinte. Două paragrafe `<p>` aflate unul sub altul într-un `<div>` sunt frați.

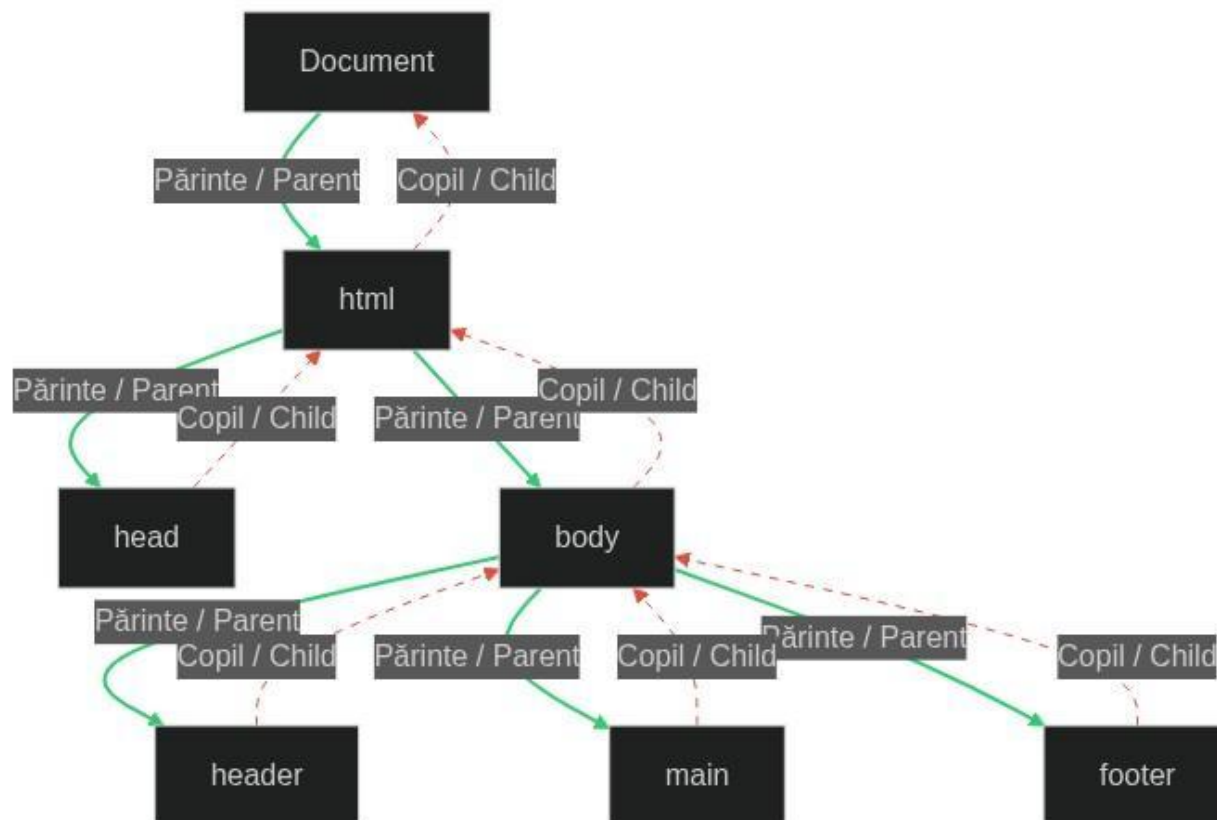


Fig. 3 - Structura ierarhică a unui document HTML. Diagrama de tip Tree Graph exemplifică relațiile de tip Părinte-Copil (Parent-Child) dintre elemente. Nodul principal (Document) conține elementul html, care se împarte în head și body. La rândul său, body găzduiește cele trei zone vizibile ale paginii: header, main și footer. Etichetele colorate evidențiază legăturile de subordonare dintre aceste noduri.

3.3 De ce este DOM-ul „Terenul de joacă” al automatizării?

Iată **Regula de Aur a Automatizării**: Roboții de testare (precum Playwright sau Selenium) **NU** au ochi. Ei nu se uită la pixelii de pe monitorul tău.

Când îi spui lui Playwright "Dă click pe butonul de Login", scriptul nu caută un dreptunghi albastru pe ecran. El intră "sub capotă", parcurge arborele DOM și caută un "nod" (element) care se potrivește cu descrierea ta (ex: id="login-btn").

Dacă un element există în DOM, dar este ascuns vizual prin CSS (ex: e făcut transparent), Playwright va ști că este acolo, dar îți va spune politicos: *"Elementul este în DOM, dar nu este vizibil pentru a putea da click pe el"*.

Acesta este motivul pentru care înțelegerea DOM-ului te transformă dintr-un simplu scriitor de teste într-un inginer de automatizare capabil să facă debugging.

3.4 Cum „citește” Playwright DOM-ul? (O privire sub capotă)

Unelte vechi de testare încercau să simuleze mișcarea fizică a mouse-ului pe ecran. Era un proces lent și instabil. Playwright face ceva mult mai inteligent: **se conectează direct la "sistemul nervos" al browserului** (prin ceva numit *Chrome DevTools Protocol*).

- **Viteza luminii:** Pentru că Playwright interoghează direct arborele DOM din memoria RAM a browserului, el poate găsi un element din 10.000 în câteva milisecunde.
- **Auto-Waiting (Așteptarea Inteligentă):** Playwright nu doar citește DOM-ul static. El "ascultă" evenimentele DOM-ului. Dacă îi spui să dea click pe un buton care încă nu a apărut pe pagină, el va sta și va "asculta" mutațiile DOM-ului până când butonul este "atașat" în structură, apoi așteaptă să devină "vizibil" și "interactiv", abia apoi executând click-ul.
- **Pătrunderea dincolo de suprafață:** Playwright poate forța interacțiuni. Dacă un element este acoperit de un alt div transparent (un bug comun de UI), poți folosi `element.click({ force: true })`, care îi spune lui Playwright: *"Ignoră ce este desenat pe ecran, du-te direct la nodul DOM al butonului și declanșează evenimentul JavaScript de Click pe el!"*.

3.5 Workshop Practic: Crearea scheletului extins pentru aplicația „Task Tracker”

Gata cu teoria! Să construim un **schelet HTML** mult mai bogat pentru aplicația noastră **To-Do List**. Vom adăuga formulare de selecție și un tabel, perfecte pentru a exersa automatizarea mai târziu.

Instrucțiuni pas cu pas:

- Deschide editorul tău de text (Visual Studio Code).
- În folderul **QA_Task_Tracker** creat anterior, deschide fișierul `index.html`.
- Vom înlocui conținutul vechi din `<body>` cu o interfață mult mai complexă. Observați utilizarea elementelor `<form>`, `<select>` și `<table>`, și atenția deosebită acordată atributelor `data-testid`!

Codul de copiat în interiorul `<body>`:

```
<header>
```

```

    <h1>Task Tracker Pro - QA Edition</h1>
    <p>Platforma completă de exersare a automatizării</p>
</header>

<main>
    <!-- Secțiunea de adăugare task -->
    <section class="task-input-section">
        <h2>Adaugă un Task Nou</h2>
        <form id="add-task-form">
            <label for="task-name">Nume Task:</label>
            <input type="text" id="task-name" data-testid="input-task-
name" placeholder="Ex: Scrie scenariii E2E" required>

            <label for="task-priority">Prioritate:</label>
            <select id="task-priority" data-testid="select-priority">
                <option value="low">Scăzută</option>
                <option value="medium" selected>Medie</option>
                <option value="high">Critică</option>
            </select>

            <label for="due-date">Termen limită:</label>
            <input type="date" id="due-date" data-testid="input-due-
date">

            <button type="submit" id="add-task-btn" data-
testid="submit-new-task">Salvează Task</button>
        </form>
    </section>

    <!-- Secțiunea de afișare task-uri (Tip Listă) -->
    <section class="task-list-section">
        <h2>Task-uri Active</h2>
        <ul id="active-tasks-list">
            <li class="task-item" data-task-status="pending">
                <input type="checkbox" class="complete-checkbox" data-
testid="check-task-1">
                <span>Învață arhitectura DOM</span>
                <span class="badge priority-high">Critică</span>
                <button class="delete-btn" data-testid="delete-task-
1">Șterge</button>
            </li>
        </ul>
    </section>

```



```

<!-- Secțiunea de istoric (Tip Tabel) -->
<section class="task-history-section">
  <h2>Istoric Task-uri Finalizate</h2>
  <table id="history-table" border="1">
    <thead>
      <tr>
        <th>ID</th>
        <th>Nume Task</th>
        <th>Data Finalizării</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>#1001</td>
        <td>Setare Boilerplate HTML</td>
        <td>09-Aug-2026</td>
      </tr>
    </tbody>
  </table>
</section>
</main>

<footer>
  <p>(c) 2026 QualiAdept Bootcamp</p>
</footer>

```

- **Testarea vizuală:** Salvează fișierul și dă refresh în Chrome.

Ce am realizat?

Am creat o provocare reală pentru QA! Acum avem:

- Un **Formular** (<form>) care poate testa comportamentul nativ al browserului.
- Un **Dropdown** (<select>) care necesită comenzi speciale în Playwright (locator.selectOption()).
- Un selector de **Data** (<input type="date">).
- Un **Tabel** de date, perfect pentru a exersa iterarea prin liste și rânduri (ex: găsirea unui text

pe coloana a 2-a dintr-un tabel).

3.6 Știați că...?

💡 Știați că...?

- **DOM-ul se actualizează "Live":** Spre deosebire de fișierul static index.html de pe hard disk-ul tău, DOM-ul se poate modifica constant după încărcarea paginii. Dacă folosești tab-ul "Elements" din DevTools, vezi DOM-ul *curent*, care poate conține zeci de elemente adăugate dinamic de JavaScript, pe care nu le-ai vedea dacă ai da doar click dreapta -> "View Page Source".
- **Atributul "data-testid" nu face nimic vizual:** Un developer poate adăuga orice atribut dorește inventând cuvântul data- în față. data-testid nu schimbă culoarea sau comportamentul elementului, este pur și simplu un far pus de developer pe care senzorii Playwright îl caută automat folosind page.getByTestId().

3.7 Poveste Aplicată: „Arborele Genealogic al Familiei Web”

💭 Să ne imaginăm DOM-ul ca pe o familie tradițională.

- Bunicul suprem este **Documentul**.
- Bunicul are doi copii: **<head>** (copilul introvertit, creierul familiei, care stă ascuns și se gândește la metadata) și **<body>** (copilul extravertit, vizibil, care interacționează cu lumea).
- În **<body>** locuiesc nepoții: Titlul **<header>**, Zona Principală **<main>** și Subsolul **<footer>**. Toți aceștia trei sunt Frați.

Dacă îi spui lui Playwright: "Găsește-mi un buton", s-ar putea să existe și în Formular, și în Lista de Task-uri. Pentru a fi precis, trebuie uneori să îi spui: "Mergi la Părintele **<form>**, iar în interiorul lui găsește-mi Copilul **<button>**". Această "rută" prin familia DOM-ului se numește *Cale de Bază (DOM Traversal)*.

3.8 Exerciții Practice

Exercițiul 1: Inspectează noile elemente (DevTools)

- Deschide noul tău index.html în Chrome și apasă F12.
- Inspectează elementul `<select id="task-priority">`.
- Extinde-l din săgeată.
- *Întrebare:* Observi tag-urile `<option>` din interiorul lui? Ele sunt Părinții, Copiii sau Frații tag-ului `<select>`?

Exercițiul 2: Adaugă o funcționalitate nouă în tabel

Tabelul nostru de Istoric are momentan doar un rând cu date (în interiorul `<tbody>`).

Sarcina ta: Du-te în fișierul index.html și adaugă un al doilea rând în tabel (un nou tag `<tr>`), cu următoarele date: ID-ul "#1002", Numele "Înțelegere Arhitectură Client-Server", și Data "Astăzi".

Salvează și verifică în browser.

3.9 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1:

Tag-urile `<option>` stau direct în interiorul tag-ului `<select>`, deci ele sunt **Copiii (Children)** tag-ului `<select>` (iar tag-ul `<select>` este Părintele lor). În Playwright, când vrei să extragi toate opțiunile dintr-un dropdown, îi vei cere robotului să numere toți "copiii" aceluia `<select>`.

Soluție Exercițiul 2:

Noul tău interior pentru eticheta `<tbody>` ar trebui să arate așa:

```
<tbody>
  <tr>
    <td>#1001</td>
    <td>Setare Boilerplate HTML</td>
    <td>09-Aug-2026</td>
  </tr>
  <!-- Rândul adăugat de tine: -->
  <tr>
    <td>#1002</td>
    <td>Înțelegere Arhitectură Client-Server</td>
    <td>10-Aug-2026</td>
  </tr>
```

```
</tbody>
```

Dacă tabelul s-a extins corect în browser, ești capabil să manipulezi independent o structură HTML validă!

Capitolul 4: Memoria Browserului, Metode HTTP și Tema Sesiunii 1

Obiective de învățare

🎯 La finalul acestui capitol, vei fi capabil să:

- Faci diferența între un request de tip GET și unul de tip POST și să știi când este folosit fiecare.
- Inspectezi tab-ul "Application" din Chrome DevTools pentru a găsi și șterge cookie-uri sau token-uri (esențial pentru testarea fluxurilor de Login/Logout).
- Realizezi prima ta temă independentă: o pagină de Autentificare structurată perfect pentru automatizare.

- Folosești platforma oficială de validare QualiAdept pentru a-ți verifica munca și a-ți debloca progresul.

4.1 Metode HTTP: GET vs. POST (Ce se întâmplă cu datele din formular?)

În Capitolul 3, am construit un <form> pentru adăugarea de task-uri. Dar când utilizatorul apasă "Salvează Task", cum ajung acele date la server? Folosind metodele HTTP. Cele mai întâlnite două metode sunt:

- **GET (Cere informații):** Este metoda implicită a browserului. Când scrii emag.ro în browser, faci un GET. Dacă ai un formular de Căutare (Search) care folosește GET, datele căutate vor apărea direct în URL (ex: site.ro/search?q=laptop).
 - *Regula QA:* NICIODATĂ nu trimitem parole sau date sensibile prin GET, deoarece rămân vizibile în istoricul browserului!
- **POST (Trimite informații):** Este folosit pentru a trimite date "ascunse" în corpul cererii (Body-ul HTTP), nu în URL. Formularul nostru de "Adaugă Task" sau orice formular de "Login/Register" trebuie să folosească POST.
 - *Regula QA:* Ca Automation Engineer, vom scrie teste de API (în modulele viitoare) în care vom simula exact aceste request-uri POST, trimițând fișiere JSON (datele) direct către server, ocolind interfața grafică!

4.2 Memoria Browserului: Cookies, Local Storage și Session Storage

Când te loghezi pe Facebook, închizi browserul și a doua zi îl deschizi iar, ești în continuare logat. Cum știe Facebook cine ești, dacă HTTP-ul este un protocol "Stateless" (fără memorie)?

Răspunsul: Serverul îți dă o "ștampilă" (un Token sau Cookie), iar browserul tău o stochează într-un sertar.

Există trei sertare principale pe care un QA trebuie să știe să le deschidă:

- **Cookies:** Bucăți mici de text. Sunt trimise automat înapoi la server la fiecare click pe care îl faci pe site. Sunt folosite clasic pentru menținerea sesiunii de login.

- **Local Storage:** Un "hard disk" virtual în browserul tău, oferit fiecărui site. Datele de aici rămân chiar dacă închizi PC-ul, până când sunt șterse explicit (manual sau prin cod). Aplicațiile moderne stochează aici "Bearer Tokens" (coduri lungi de securitate).
- **Session Storage:** Similar cu Local Storage, dar cu memorie scurtă. Dacă ai închis tab-ul, datele de aici se șterg instant.

4.3 DevTools: Tab-ul Application (Arma secretă a QA-ului)

Cea mai mare greșeală a unui tester (manual sau automat) care testează funcționalitatea de Login este să dea click pe butonul de "Logout" și să creadă că a testat totul. "Logout-ul" doar șterge cookie-ul vizual. Dar ce se întâmplă dacă ștergi cookie-ul manual din DevTools? Mai ești logat?

Pentru a investiga aceste memorii, Chrome ne oferă tab-ul **Application**.

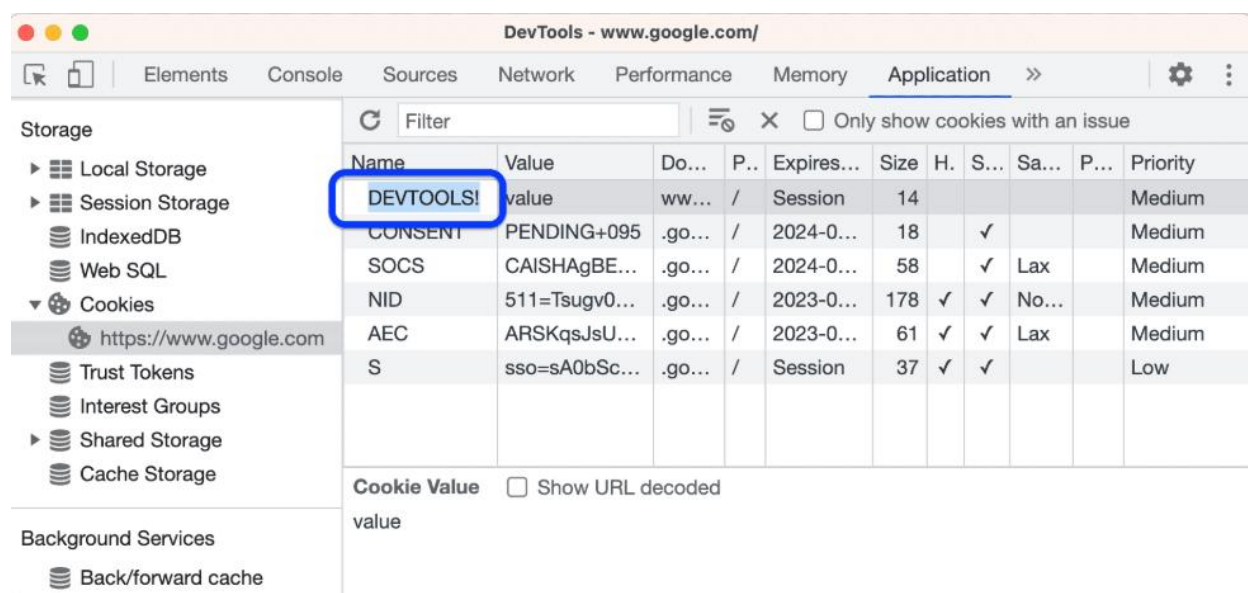


Fig. 4 - Panoul Chrome DevTools, având selectat tab-ul "Application" din meniul de sus, iar în panoul din stânga fiind evidențiate subsecțiunile "Storage" -> "Local Storage" și "Cookies"

Cum va folosi Playwright această memorie?

În modulele avansate, vom învăța un truc genial: în loc ca Playwright să completeze userul și parola la fiecare test (ceea ce ar dura 3 secunde per test), îl vom pune să se logheze o singură dată, vom "fura" Cookie-ul din DevTools folosind cod, și îl vom injecta direct în Memoria

Browserului pentru restul de 100 de teste. Timp economisit: enorm!

4.4 Știați că...?

- **Cookie-urile au inventat "Coșul de Cumpărături":** Primul cookie a fost creat în 1994 de Lou Montulli (inginer la Netscape) exact pentru a permite unui magazin virtual să țină minte ce produse ai pus în coș de la o pagină la alta.
- **Limita de stocare:** Un fișier Cookie poate stoca doar 4KB de date (foarte puțin), în timp ce Local Storage poate stoca până la 5MB (suficient pentru a salva o carte întreagă în memoria browserului tău).

4.5 Poveste Aplicată: „Cartea Poștală și Coletul Blindat”

 Cea mai simplă analogie pentru HTTP GET și POST.

- **Request-ul GET este ca o Carte Poștală.** Tu scrii mesajul pe spatele ei și o dai poștașului. Pe drum, poștașul, vecinii și oricine manipulează cartea poștală (serverele de internet) pot citi mesajul tău (pentru că e vizibil în URL). Este perfect pentru a cere "Vremea la București", dar groaznic pentru a trimite "Parola mea este 1234".
- **Request-ul POST este ca un Colet Blindat.** Adresa de destinație e scrisă pe cutie, dar conținutul (Body-ul) este închis înăuntru și sigilat. Nimeni de pe traseu nu vede ce e înăuntru. Când facem un formular de Login, folosim mereu POST!

4.6 Exerciții Practice

Exercițiul 1: Vânătoarea de Cookie-uri

- Deschide Google Chrome și intră pe orice site unde ești logat (ex: YouTube, eMAG sau chiar GitHub).
- Apasă F12 pentru a deschide DevTools.
- Mergi la tab-ul **Application** (dacă nu îl vezi, apasă pe săgeata >> din meniul DevTools de sus).
- În meniul din stânga, extinde secțiunea **Cookies** și dă click pe numele site-ului.

- *Experiment:* Găsește linia care pare să fie legată de sesiune sau login (are adesea cuvinte ca "sess", "token", "auth"), dă click-dreapta pe ea și alege "Delete". Dă un Refresh paginii. Ce s-a întâmplat?

4.7 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1:

Dacă ai șters cookie-ul corect (cel de sesiune) și ai dat refresh paginii, **ai fost delogat instantaneu** de pe site, chiar dacă nu ai apăsăat butonul oficial de "Logout". Așa verificăm noi, ca ingineri QA, dacă securitatea unei aplicații este corect legată de acel Cookie și dacă serverul respinge accesul utilizatorilor care nu dețin acea "ștampilă" în memoria browserului!

4.8 Tema pentru Acasă (Proiect Sesiunea 1) & Platforma QualiAdept Cloud Evaluation

A sosit momentul să demonstrezi ce ai învățat! Acum că ai înțeles scheletul HTML și importanța atributelor (id, data-testid), vei construi prima ta pagină independentă, gândită *din start* pentru a fi testată de roboți.

Sarcina ta (Task-ul de business):

Construiește o pagină HTML de "Login și Înregistrare" pentru aplicația noastră *Task Tracker*.

Cerințe Tehnice (Acceptance Criteria):

- Documentul trebuie să aibă structura Boilerplate validă (<html>, <head>, <body>).
- Titlul paginii (în tab-ul browserului) să fie: Task Tracker Login.
- Trebuie să ai un <form> care conține:
 - Un <input> pentru Email, având id="login-email" și data-testid="input-email".
 - Un <input> pentru Parolă (type="password"), având id="login-password" și data-testid="input-password".
 - Un <button> pentru trimitere, având textul "Log In" și atributul data-testid="btn-submit-login".
- Formularul trebuie să aibă un container Părinte <div class="login-container">.

Introducere în Platforma Certify QualiAdept

Pentru a-ți valida codul și a debloca modulele viitoare, vom folosi instrumente din lumea reală a

ingineriei. Am construit pentru tine **Platforma QualiAdept Cloud Evaluation** (certify.qualiadept.eu).

Această platformă este ecosistemul tău de învățare și funcționează exact ca sistemele de integrare continuă (CI/CD) din companiile IT:

-  **Motor de Validare Automată (Static Inspection):** În momentul în care îți trimiți codul, platforma rulează instantaneu un set de teste peste structura ta HTML. Sistemul verifică determinist prezența fiecărui atribut, tag și ID necesar pentru automatizarea viitoare.
-  **Progresie Secvențială (Drip Content):** Următoarele module din curs sunt blocate cu lacăt. Singura metodă prin care poți avansa în bootcamp este să obții statusul "Passed" (Promovat) la tema curentă, rezolvând absolut toate cerințele.
-  **Portofoliu Public Verificabil:** Toate temele tale trecute cu succes se centralizează pe un profil public. La final, vei obține un link pe care îl poți pune direct în CV, dovedind recrutorilor că ești un „QualiAdept Verified QA Engineer”.

Cum te Auto-Validezi (Predarea Temei)

Pasul 1: Scrie codul tău HTML în editorul tău preferat (ex: VS Code) și asigură-te că arată corect vizual dând dublu-click pe fișierul .html în browser.

Pasul 2: Accesează platforma certify.qualiadept.eu și autentifică-te rapid și sigur cu contul tău de GitHub.

Pasul 3: În Tabloul de Bord (LMS Dashboard), localizează **Modulul 01** și apasă pe butonul **Deschide Workspace**.

Pasul 4: Copiază codul tău din VS Code, lipește-l în zona de cod din platformă și apasă pe butonul de validare (Submit).

Pasul 5 (Feedback și Validare):

- Dacă ai scris codul perfect și ai pus ancorele corecte pentru automatizare, vei primi un scor de 100% și bannerul verde cu ☒ **Promovat**. Sistemul va bifa modulul ca fiind absolvit, iar Modulul 2 se va pregăti pentru deblocare!
- Dacă validarea eșuează, nu te panica! Analizează raportul de erori returnat de platformă, vezi ce atribut sau ID ai omis, întoarce-te în editorul tău de cod, corectează și trimite codul din nou.

Mândrește-te cu succesul tău!

Odată ce ai validat tema cu succes, întoarce-te pe Dashboard-ul platformei și apasă pe butonul **"Share My Profile"**. Distribuie link-ul obținut pe grupul de Discord al comunității QualiAdept (sau chiar pe LinkedIn) pentru a te lăuda cu prima ta victorie tehnică. Ești oficial pe drumul cel bun!